Module 14 - chapter 10

Automate cleanup of old Snapshots

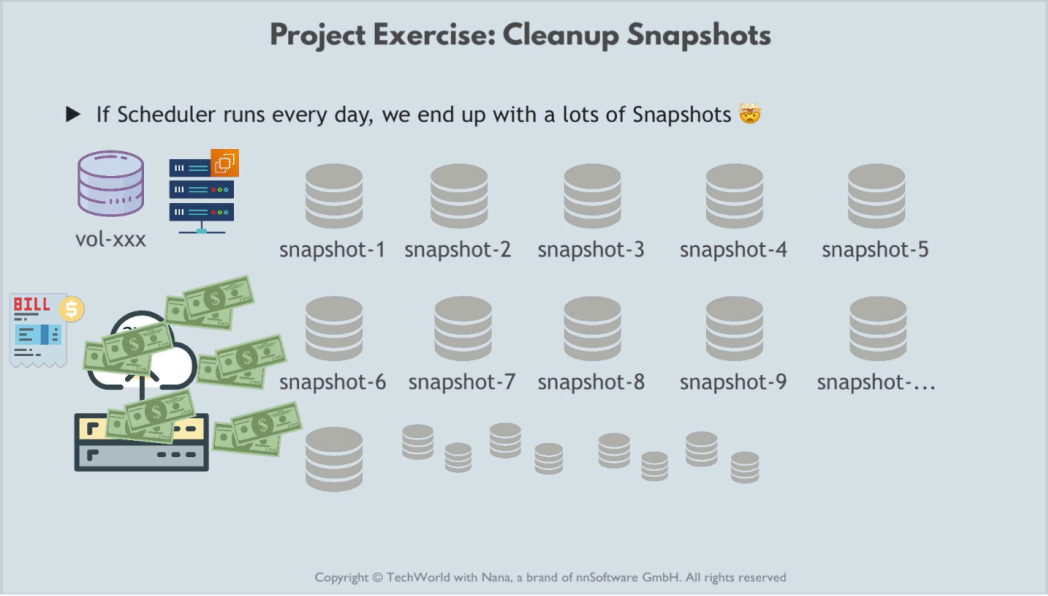
The project file used in this lecture can be found here:

- https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/cleanup-snapshots.py

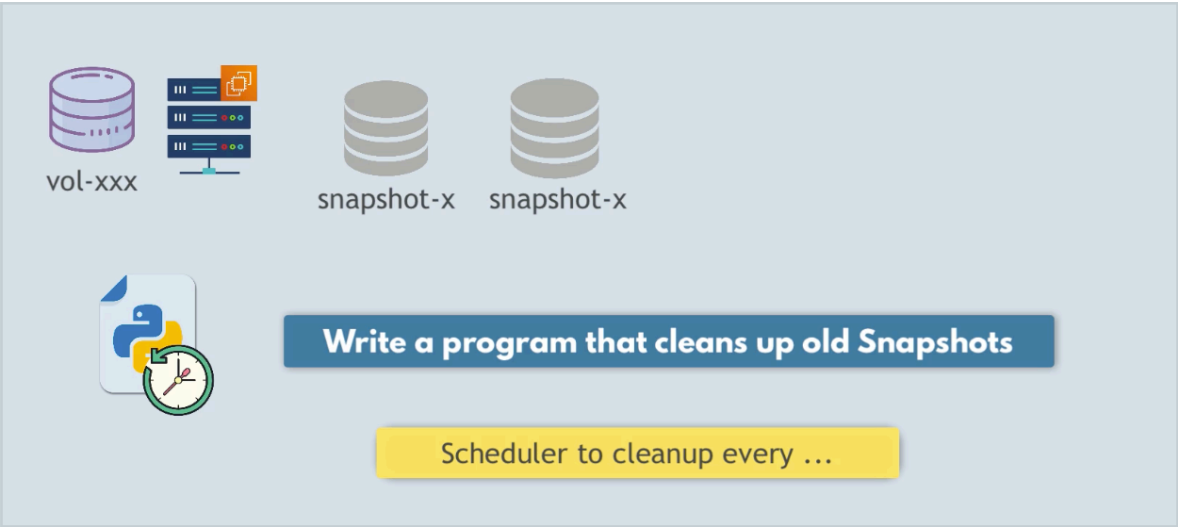
Project intro

Cleanup snapshots

If we run the scheduler every day -> lots of Snapshots at the end of the month which will demand a lot of storage



And we only really need the most recent ones, so we will write a program that cleans up old Snapshots and for it to be also an automated task using scheduler.



Implementation

1. Collect the snapshots
2. Find the date and keep the latest snapshots.

To collect the snapshots EC2 service has a function called describe_snapshots() https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe_snapshots.html

describe_snapshots() will return by default all of the snapshots including ones created by AWS and not by our python program. So we need to define the **OwnersId** to get only the snapshots that we want:

Request Syntax

```
response = client.describe_snapshots(
    MaxResults=123,
    NextToken='string',
    OwnerIds=[
        'string',
    ],
    RestorableByUserIds=[
        'string',
    ],
    SnapshotIds=[
        'string',
    ],
    DryRun=True|False,
    Filters=[
        {
            'Name': 'string',
            'Values': [
                'string',
            ]
        },
    ]
)
```

And these are the 3 possible values:

PARAMETERS:

- **MaxResults** (*integer*) – The maximum number of items to return for this request. To get the next page of items, make another request with the token returned in the output. For more information, see Pagination.
- **NextToken** (*string*) – The token returned from a previous paginated request. Pagination continues from the end of the items returned by the previous request.
- **OwnerIds** (*list*) –

  Scopes the results to snapshots with the specified owners. You can specify a combination of Amazon Web Services account IDs, `self`, and `amazon`.

  - *(string)* –
- **RestorableByUserIds** (*list*) –

  The IDs of the Amazon Web Services accounts that can create volumes from the snapshot.

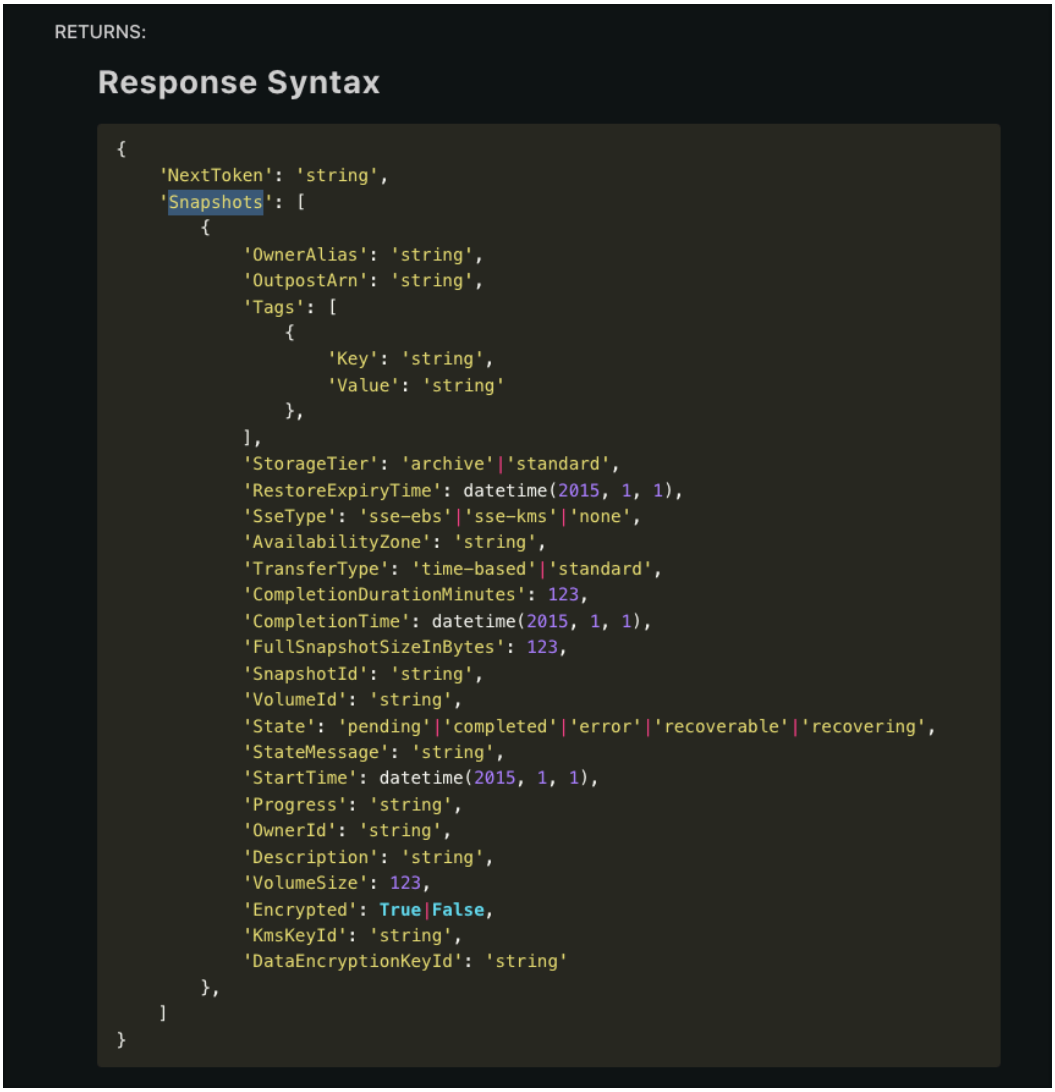
  - *(string)* –

And the ones created by us are -> ‘self’

cleanup-snapshots.py

```python
import boto3

ec2_client = boto3.client( *args: 'ec2', region_name="eu-west-2")

snapshots =ec2_client.describe_snapshots(
    OwnerIds=['self']
)
```

And the structure of the response is:



RETURNS:

Response Syntax

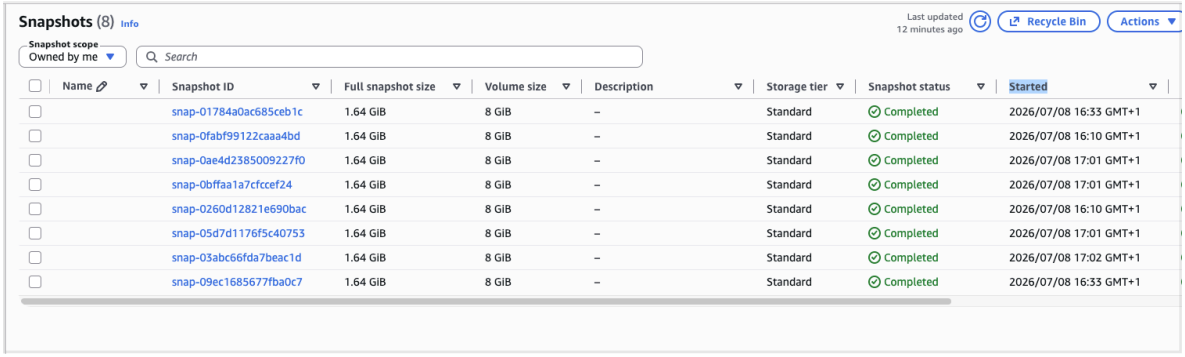
```
{
    'NextToken': 'string',
    'Snapshots': [
        {
            'OwnerAlias': 'string',
            'OutpostArn': 'string',
            'Tags': [
                {
                    'Key': 'string',
                    'Value': 'string'
                },
            ],
            'StorageTier': 'archive'|'standard',
            'RestoreExpiryTime': datetime(2015, 1, 1),
            'SseType': 'sse-ebs'|'sse-kms'|'none',
            'AvailabilityZone': 'string',
            'TransferType': 'time-based'|'standard',
            'CompletionDurationMinutes': 123,
            'CompletionTime': datetime(2015, 1, 1),
            'FullSnapshotSizeInBytes': 123,
            'SnapshotId': 'string',
            'VolumeId': 'string',
            'State': 'pending'|'completed'|'error'|'recoverable'|'recovering',
            'StateMessage': 'string',
            'StartTime': datetime(2015, 1, 1),
            'Progress': 'string',
            'OwnerId': 'string',
            'Description': 'string',
            'VolumeSize': 123,
            'Encrypted': True|False,
            'KmsKeyId': 'string',
            'DataEncryptionKeyId': 'string'
        },
    ]
}
```

So we can print:

```
snapshots =ec2_client.describe_snapshots(
    OwnerIds=['self']
)

print(snapshots['Snapshots'])   You, Moments ago •
```

So we have a list of snapshots.

Now let's get the time of creation of each snapshot using the 'StartTime' key from the response which represents the column 'Started' in AWS console




```
'SnapshotId': 'string',
'VolumeId': 'string',
'State': 'pending'|'completed'|'error'|'recoverable'|'recovering',
'StateMessage': 'string',
'StartTime': datetime(2015, 1, 1),
'Progress': 'string',
'OwnerId': 'string',
'Description': 'string',
```

And we want to sort the dates.

How do we sort a list by a value of a dictionary? So I have a list of 8 dictionaries, and each dictionary has a date and we want to sort the list of dictionaries by the date -> the StartTime

-> sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'))

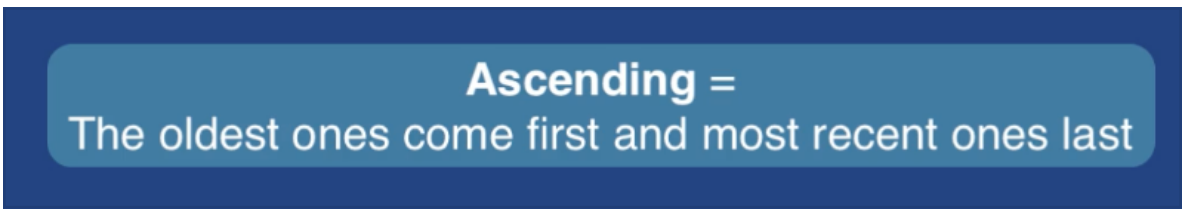
We print the unsorted list and the sorted list

```
sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'))

# print unsorted list
for snap in snapshots['Snapshots']:
    print(snap['StartTime'])

print('################')

# print sorted list   You, Moments ago • Uncommitted changes
for snap in sorted_by_date:
    print(snap['StartTime'])

```

We can see the first list is unsorted, random positions

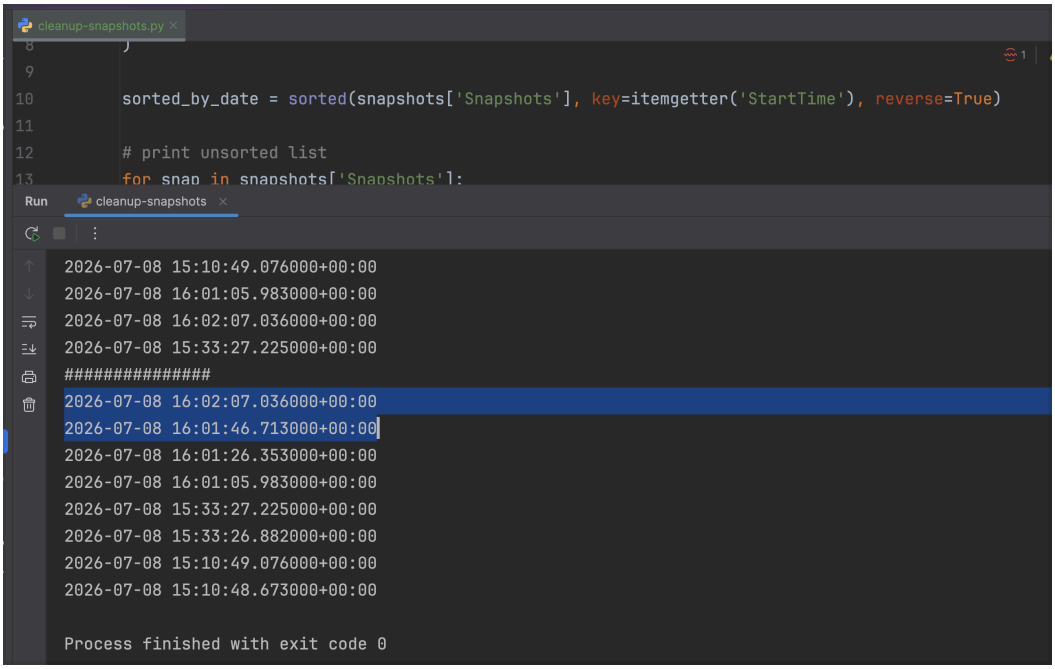
```
/Users/astrid/PythonProject/Automation_
2026-07-08 15:33:26.882000+00:00
2026-07-08 15:10:48.673000+00:00
2026-07-08 16:01:26.353000+00:00
2026-07-08 16:01:46.713000+00:00
2026-07-08 15:10:49.076000+00:00
2026-07-08 16:01:05.983000+00:00
2026-07-08 16:02:07.036000+00:00
2026-07-08 15:33:27.225000+00:00
################
2026-07-08 15:10:48.673000+00:00
2026-07-08 15:10:49.076000+00:00
2026-07-08 15:33:26.882000+00:00
2026-07-08 15:33:27.225000+00:00
2026-07-08 16:01:05.983000+00:00
2026-07-08 16:01:26.353000+00:00
2026-07-08 16:01:46.713000+00:00
2026-07-08 16:02:07.036000+00:00
```

And the second list is sorted -> Ascending



If we want Descending sorting then we add a parameter to the sorted() call called ‘reverse’

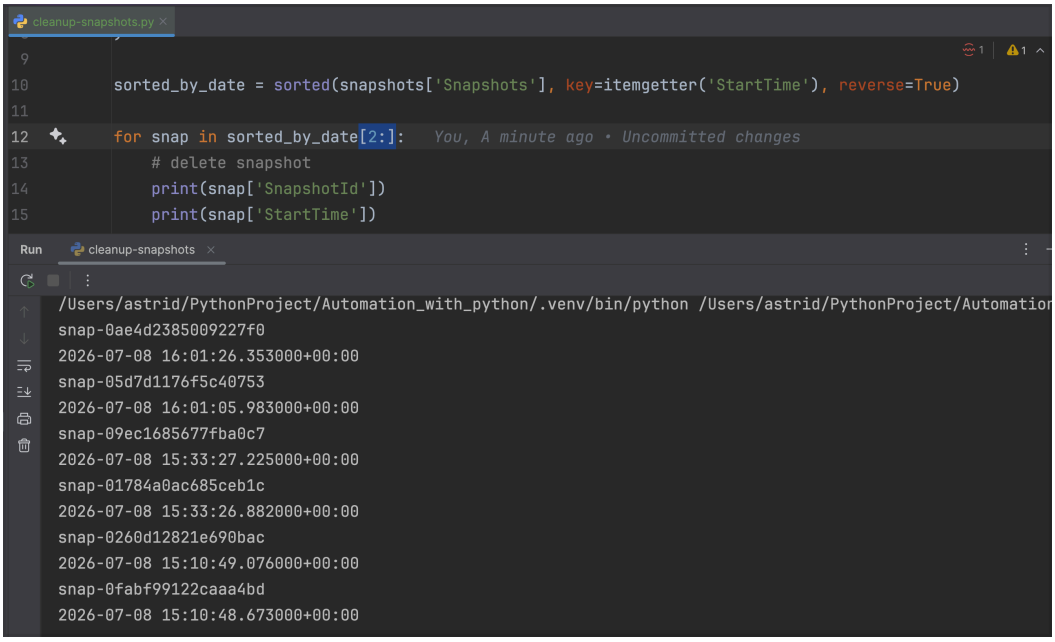
-> sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)



Now we have the more recent snapshots at the top of the sorted list.

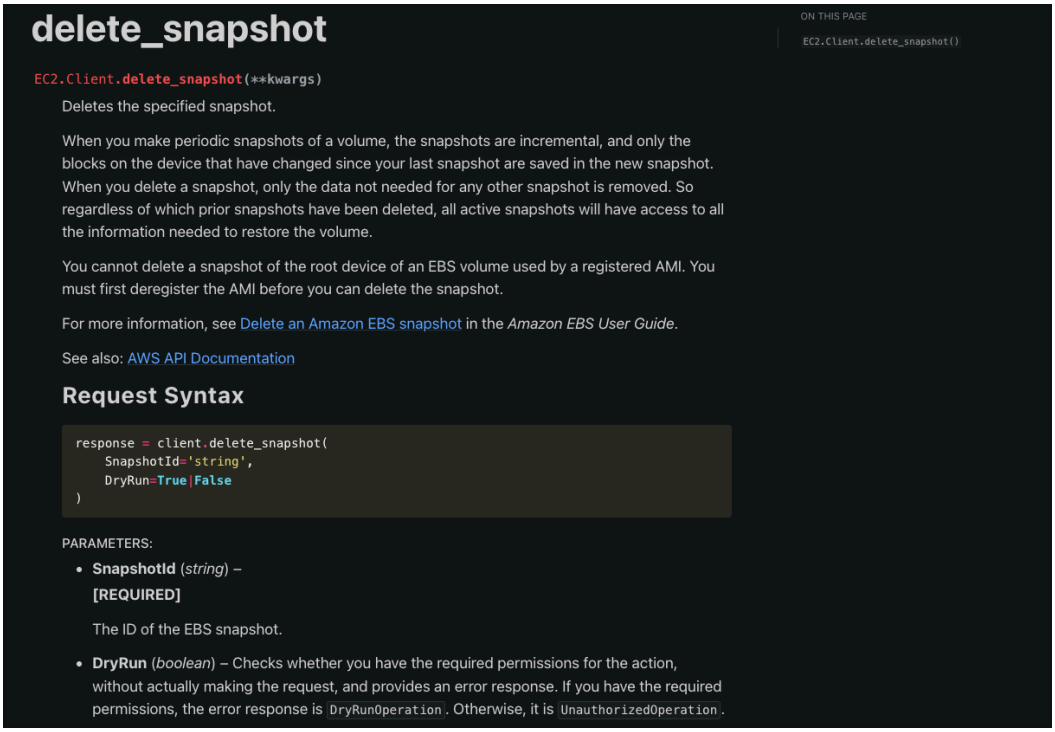
Now we will iterate over the list and remove old snapshots starting at the third element as we want to keep the first 2, the most recent ones.

-> sorted_by_date[2:] start at index 2 until the end [2:]



And the first 2 snapshots of the list are not there, 6 were printed instead of 8.

Now we can delete the snapshots using delete_snapshot() https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/delete_snapshot.html



delete_snapshot

ON THIS PAGE

EC2.Client.delete_snapshot()

EC2.Client.delete_snapshot(**kwargs)

Deletes the specified snapshot.

When you make periodic snapshots of a volume, the snapshots are incremental, and only the blocks on the device that have changed since your last snapshot are saved in the new snapshot. When you delete a snapshot, only the data not needed for any other snapshot is removed. So regardless of which prior snapshots have been deleted, all active snapshots will have access to all the information needed to restore the volume.

You cannot delete a snapshot of the root device of an EBS volume used by a registered AMI. You must first deregister the AMI before you can delete the snapshot.

For more information, see Delete an Amazon EBS snapshot in the *Amazon EBS User Guide*.

See also: AWS API Documentation

Request Syntax

```
response = client.delete_snapshot(
    SnapshotId='string',
    DryRun=True|False
)
```

PARAMETERS:

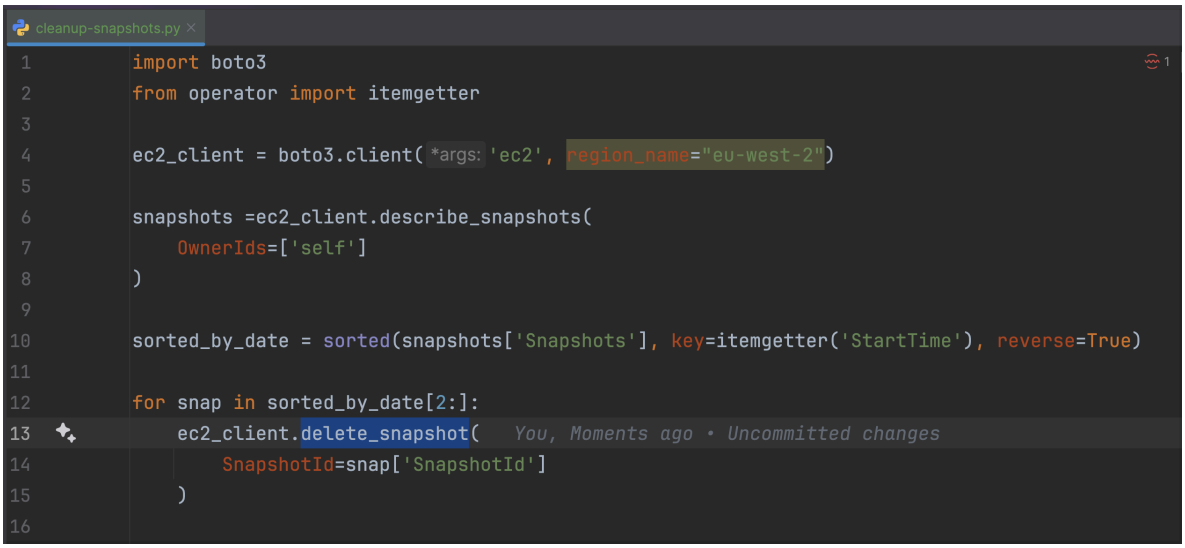
- **SnapshotId** (*string*) –
  [REQUIRED]

  The ID of the EBS snapshot.

- **DryRun** (*boolean*) – Checks whether you have the required permissions for the action, without actually making the request, and provides an error response. If you have the required permissions, the error response is `DryRunOperation`. Otherwise, it is `UnauthorizedOperation`.

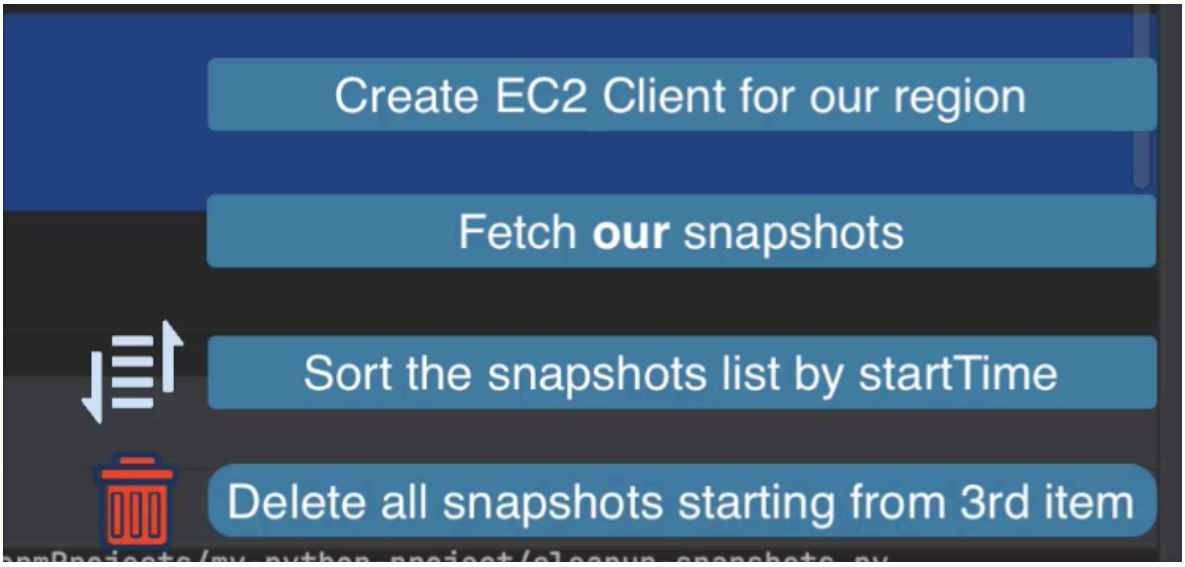
And snapshotId is required.

```
for snap in sorted_by_date[2:]:
    ec2_client.delete_snapshot(
        SnapshotId=snap['SnapshotId']
    )
```

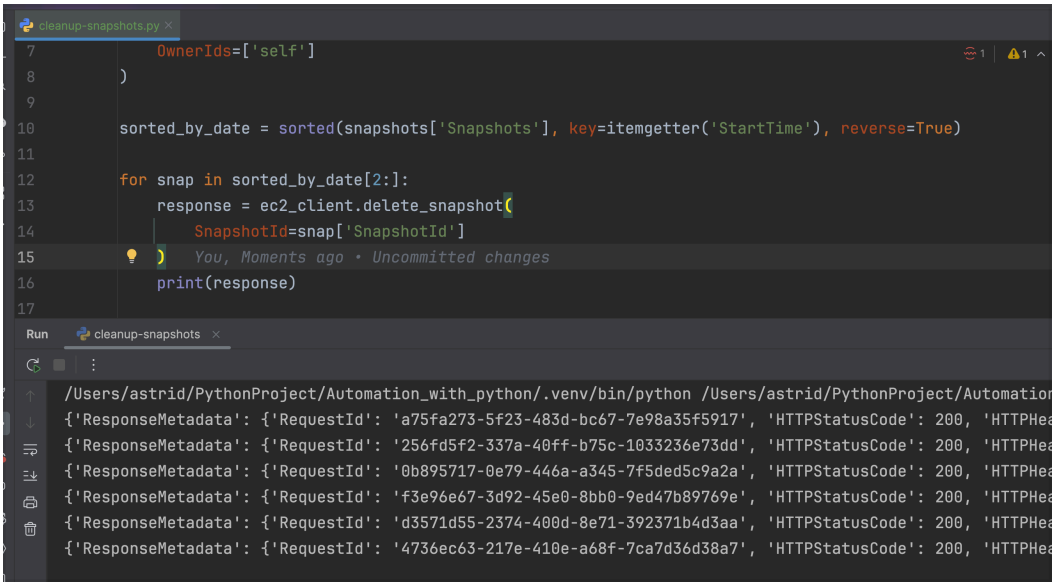


cleanup-snapshots.py

```
1   import boto3
2   from operator import itemgetter
3
4   ec2_client = boto3.client( *args: 'ec2', region_name="eu-west-2")
5
6   snapshots =ec2_client.describe_snapshots(
7       OwnerIds=['self']
8   )
9
10  sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)
11
12  for snap in sorted_by_date[2:]:
13      ec2_client.delete_snapshot(   You, Moments ago • Uncommitted changes
14          SnapshotId=snap['SnapshotId']
15      )
16
```

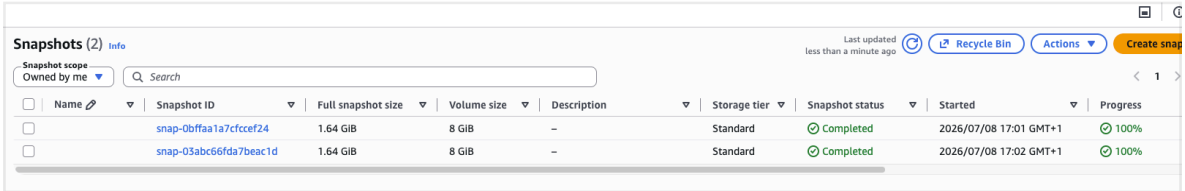


And now let's execute the program



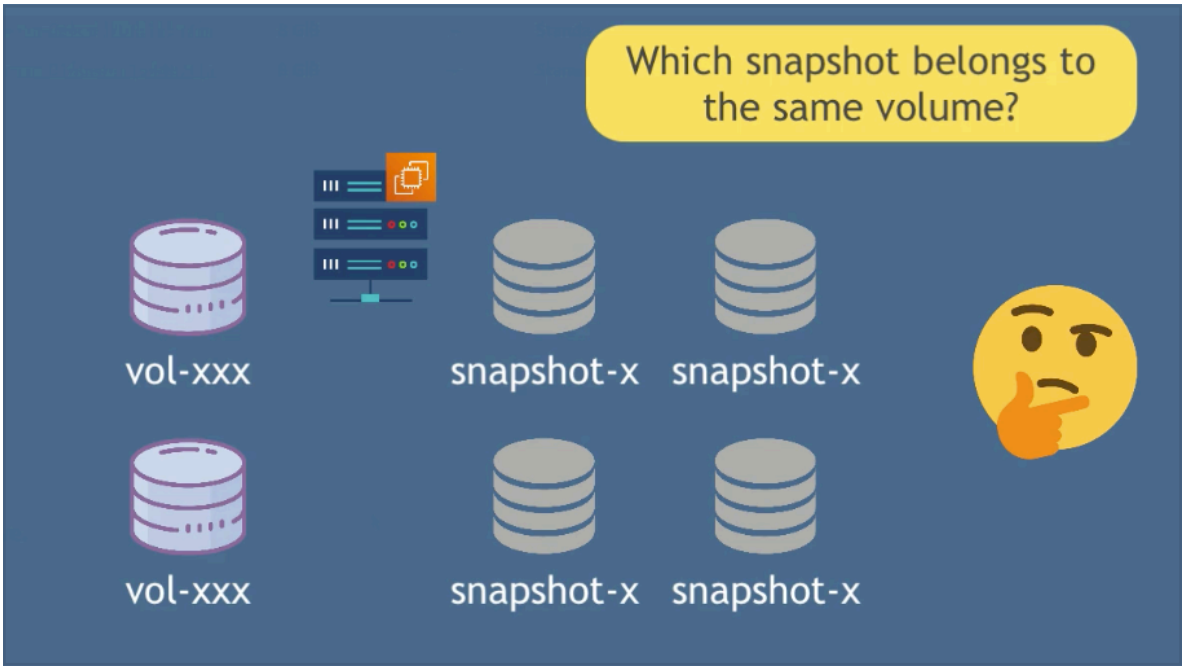
6 snapshots deleted 👆

2 snapshots remain 👇



Scheduler

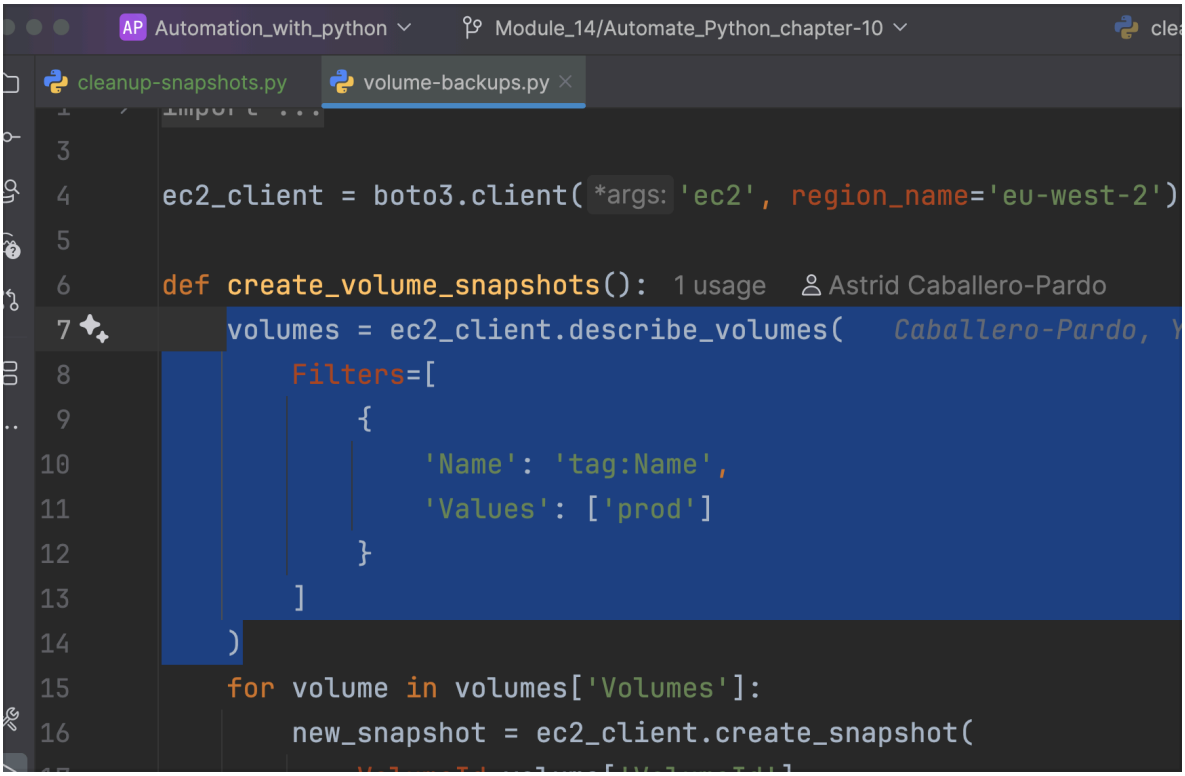
This script should run every week or month so now let's add the scheduler.



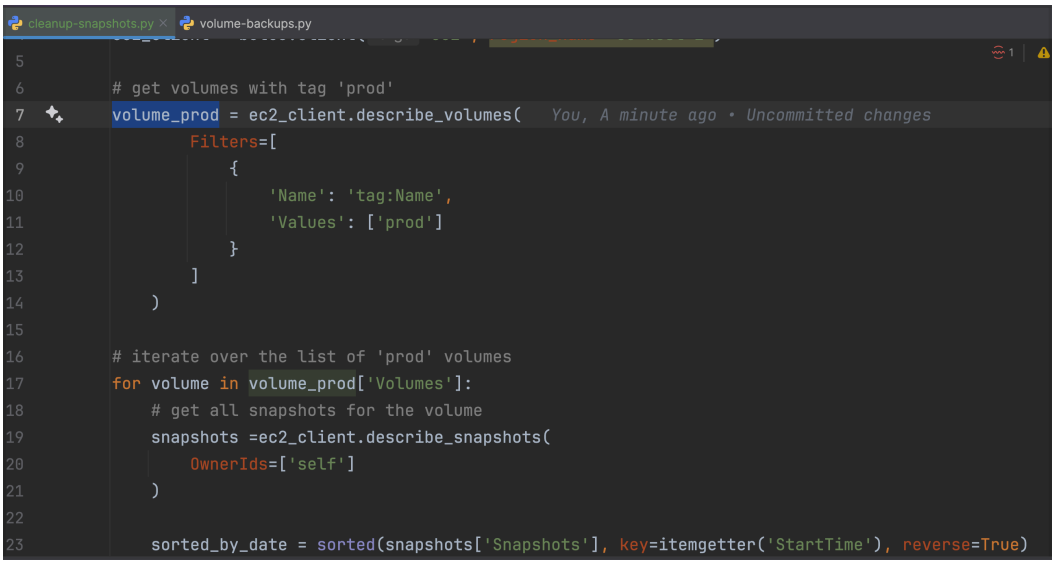
So we will filter the snapshots by specific volume, and for that we need to list the volumes that we have

```
volume_prod = ec2_client.describe_volumes(
    Filters=[
        {
            'Name': 'tag:Name',
            'Values': ['prod']
        }
    ]
)
```

We created ‘prod’ server and we made snapshots of the server with tag ‘prod’ in Module 14 - chapter 9 with the python script ‘volume-backups.py’



So now the code goes like this:



And we also need to specify the Volume-Id to filter the snapshots not only the ‘OwnerIds’

Request Syntax

```
response = client.describe_snapshots(
    MaxResults=123,
    NextToken='string',
    OwnerIds=[
        'string',
    ],
    RestorableByUserIds=[
        'string',
    ],
    SnapshotIds=[
        'string',
    ],
    DryRun=True|False,
    Filters=[
        {
            'Name': 'string',
            'Values': [
                'string',
            ]
        },
    ]
)
```

And ‘volume-id’ is one of the filters:

- **Filters** (*list*) –

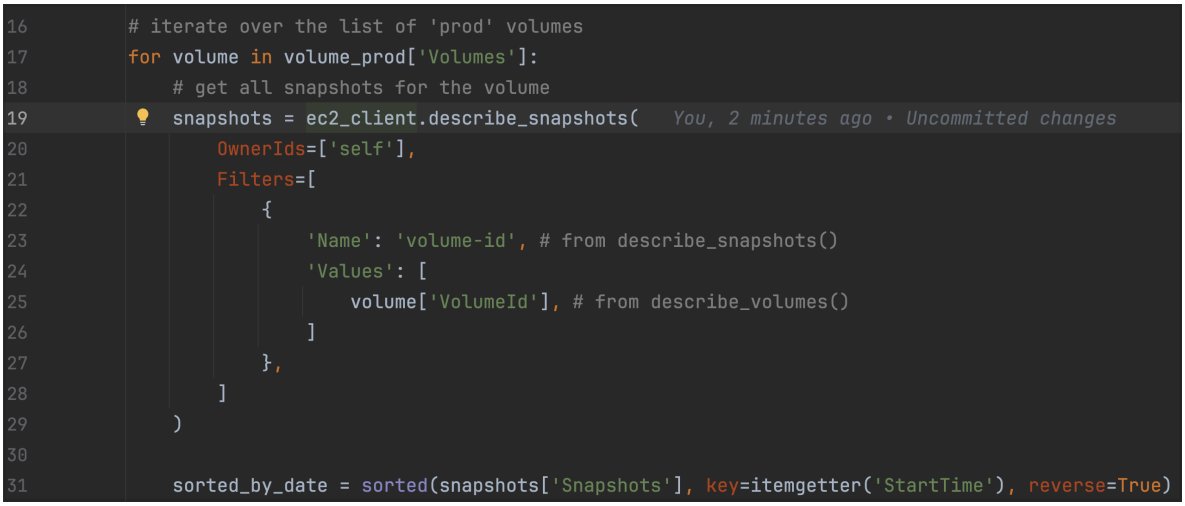
  The filters.

  - `description` - A description of the snapshot.
  - `encrypted` - Indicates whether the snapshot is encrypted (`true` | `false`)
  - `owner-alias` - The owner alias, from an Amazon-maintained list (`amazon`). This is not the user-configured Amazon Web Services account alias set using the IAM console. We recommend that you use the related parameter instead of this filter.
  - `owner-id` - The Amazon Web Services account ID of the owner. We recommend that you use the related parameter instead of this filter.
  - `progress` - The progress of the snapshot, as a percentage (for example, 80%).
  - `snapshot-id` - The snapshot ID.
  - `start-time` - The time stamp when the snapshot was initiated.
  - `status` - The status of the snapshot (`pending` | `completed` | `error`).
  - `storage-tier` - The storage tier of the snapshot (`archive` | `standard`).
  - `transfer-type` - The type of operation used to create the snapshot (`time-based` | `standard`).
  - `tag` :<key> - The key/value combination of a tag assigned to the resource. Use the tag key in the filter name and the tag value as the filter value. For example, to find all resources that have a tag with the key `Owner` and the value `TeamA` , specify `tag:Owner` for the filter name and `TeamA` for the filter value.
  - `tag-key` - The key of a tag assigned to the resource. Use this filter to find all resources assigned a tag with a specific key, regardless of the tag value.
  - `volume-id` - The ID of the volume the snapshot is for.
  - `volume-size` - The size of the volume, in GiB.
  - *(dict)* –

So I copy the Filter parameter

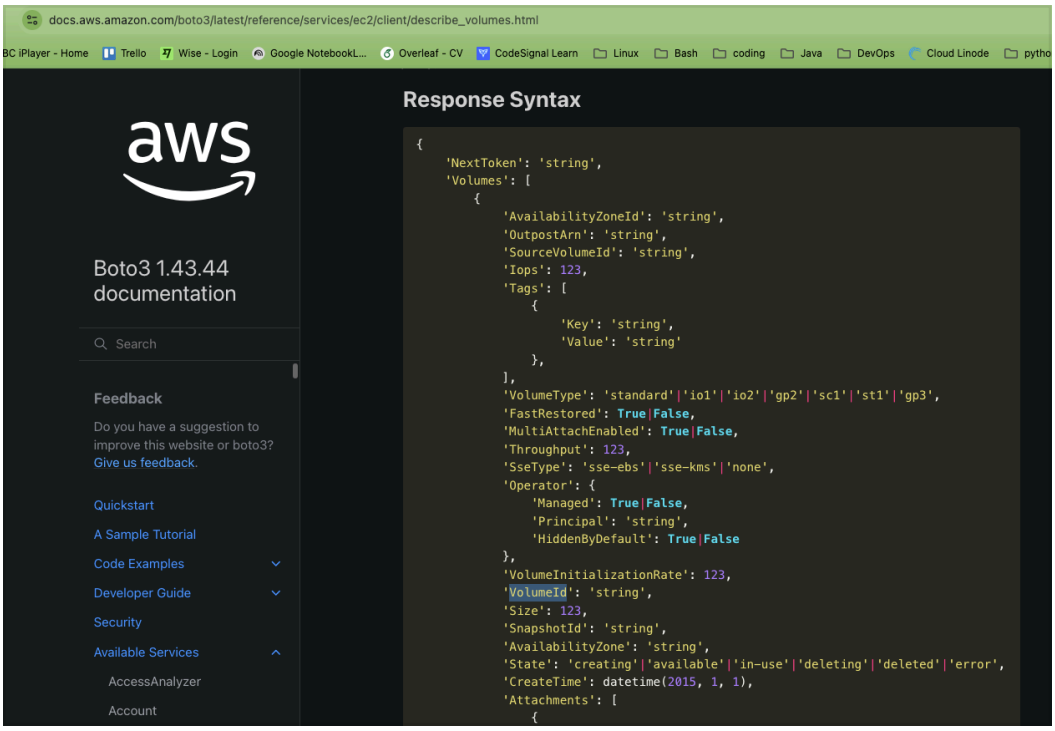
```
15
16      # iterate over the list of 'prod' volumes
17      for volume in volume_prod['Volumes']:
18          # get all snapshots for the volume
19          snapshots =ec2_client.describe_snapshots(
20              OwnerIds=['self'],
21              Filters=[
22                  {
23                      'Name': 'string',
24                      'Values': [
25                          'string',
26                      ]
27                  },
28              ]   You, Moments ago • Uncommitted changes
29          )
30
31          sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)
32
```

And I edit it with the filter I want to use, see line 23 👇



```
16      # iterate over the list of 'prod' volumes
17      for volume in volume_prod['Volumes']:
18          # get all snapshots for the volume
19          snapshots = ec2_client.describe_snapshots(   You, 2 minutes ago • Uncommitted changes
20              OwnerIds=['self'],
21              Filters=[
22                  {
23                      'Name': 'volume-id', # from describe_snapshots()
24                      'Values': [
25                          volume['VolumeId'], # from describe_volumes()
26                      ]
27                  },
28              ]
29          )
30
31          sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)
```

describe_volume() returns ‘VolumeId’ 👆 line 25



And this is the code:

```python
    # iterate over the list of 'prod' volumes
    for volume in volume_prod['Volumes']:
        # get all snapshots for the specific volume
        snapshots = ec2_client.describe_snapshots(
            OwnerIds=['self'],   You, 2 minutes ago • Uncommitted changes
            Filters=[
                {
                    'Name': 'volume-id', # from describe_snapshots()
                    'Values': [
                        volume['VolumeId'], # from describe_volumes()
                    ]
                },
            ]
        )

        sorted_by_date = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)

        # delete the snapshot
        for snap in sorted_by_date[2:]:
            response = ec2_client.delete_snapshot(
                SnapshotId=snap['SnapshotId']
            )
            print(response)
```